

codehive

The `__init__()` Method

Mastering the Python Class Constructor

1. What is `__init__`?

The `__init__()` method is a special, built-in function that all Python classes possess. Technically known as a **Constructor**, it is triggered automatically the moment an object is created from a class.

Its primary purpose is to **initialize** (assign values to) the data members of the class when an object is instantiated.

codehive

2. Basic Syntax

The name starts and ends with double underscores (dunder), signifying it is a special method in Python.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Emil", 36)
```

In this example, when `Person("Emil", 36)` is called, the `__init__` method runs instantly, setting `Emil` to `self.name`.

3. The 'Self' Connection

You will notice the **self** parameter in the method definition. This is a reference to the current instance of the class.

It is used to access variables that belong to the class. Without `self`, the program wouldn't know which object's name you are trying to set.

Note: We will dive deeper into 'self' in the next module.

codehive

4. Why use `__init__`?

Compare these two approaches to understand the efficiency of the constructor:

WITHOUT `__init__`

Manual assignment for every instance.

```
p1.name = "John"  
p1.age = 30
```

WITH `__init__`

One-line creation with data.

```
p1 = Person("John", 30)
```

The constructor ensures every object created has the necessary data to function correctly from its birth.

5. Default Values

You can provide **Default Values** for parameters in the `__init__` method. This makes certain arguments optional.

```
class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age

p1 = Person("Emil") # age defaults to 18
```

In the case of `p1`, we only provided a name, so Python automatically assigned 18 to the age.

6. Multiple Parameters

Your constructor can be as complex as needed to represent real-world objects.

```
class Car:
    def __init__(self, brand, model, year, color):
        self.brand = brand
        self.model = model
        self.year = year
        self.color = color

c1 = Car("Tesla", "Model S", 2023, "Red")
```

7. Automatic Execution

Remember: You **never** call `__init__()` directly. Python handles the call in the background.

Standard Workflow

1. User calls `Class()`
2. Python allocates memory
3. Python calls `__init__` automatically
4. Object is ready for use

8. Handling Logic in Init

Beyond just assigning variables, `__init__` can perform startup logic, like opening a database connection or calculating values.

```
class Rectangle:
    def __init__(self, w, h):
        self.width = w
        self.height = h
        self.area = w * h # Immediate calculation
```

9. Knowledge Check

Exercise

What is the purpose of the `__init__()` method in Python classes?

☐ To delete an object

☒ **To assign initial values to object properties**

☐ To print object information

10. Summary Checklist

- ✓ `__init__` is the Class Constructor.
- ✓ It runs automatically on instantiation.
- ✓ It requires `self` as the first parameter.
- ✓ It serves to set the initial state of an object.
- ✓ It supports default parameter values.

codehive

Master the Internals of OOP